MA22037 Coursework (2025/26)

The coursework counts for 25% of the final grade.

You are expected to upload your final report to Moodle as a **single document**, containing both the code and the mathematical writeup. Please submit two versions of your report: one as a .pynb file and one as a .pdf file. Please see the markdown tutorial for instructions on how to export your Python notebook to pdf.

You are expected to type up your solutions. Handwritten solutions are not accepted.

The online session on Friday 20th March will focus on how to type mathematics using markdown in Python notebooks. In addition, you are welcome to email the lecturer (James Foster, jmf68@bath.ac.uk) or come to his office if you would like further advice on typing mathematics.

Due: Wednesday 15th April at 5pm

The coursework will be graded as follows:

- [60%] Content.
- [40%] Presentation.

Each question in the assignment is equally weighted (although not all questions are of the same length). There are 50 marks in total.

All questions in the assignment must be fully typed up. Any handwritten submission risks losing presentation marks.

Section A

Fourier series are an important alternative to polynomials in approximating functions. Any piecewise-continuous real-valued function $u : [0, 1] \to \mathbb{R}$ can be expressed as

$$u(x) = \sum_{k=-\infty}^{\infty} u_k \, e^{2\pi \, k \, \mathrm{i} \, x},$$

for some $u_k \in \mathbb{C}$ known as *Fourier coefficients*, and the sum converges in the sense that

$$\int_0^1 (u(x) - \tilde{u}_J(x))^2 \, dx \to 0 \qquad \text{as } J \to \infty,$$

where $\tilde{u}_J(x) := \sum_{k=-J}^{J} u_k \, e^{2\pi \, k \, \mathrm{i} \, x}$ is known as a *trigonometric polynomial*.

A1 [5 marks]

Show that

$$\int_0^1 |u(x)|^2 \, dx = \sum_{k=-\infty}^{\infty} |u_k|^2$$

and that

$$u_k = \int_0^1 u(x)\, e^{-2\pi\, k\, \mathrm{i}\, x}\, dx.$$

***Hint 1**: First show that $\int_0^1 e^{2\pi\,(j-k)\,\mathrm{i}\,x}\, dx = 0$ for distinct integers j, k and integrate the square $|u|^2 = u\bar{u}$.*

***Hint 2**: For the second part, multiply the series expression for $u(x)$ by $e^{-2\pi\, k\, \mathrm{i}\, x}$ and integrate over $[0, 1]$.*

A2 [5 marks]

Consider the step function $u : [0, 1] \to \mathbb{R}$ defined by

$$u(x) = \begin{cases} 0, & x < 1/2, \\ 1, & x \geq 1/2. \end{cases}$$

Determine the coefficients u_k such that $u(x) = \sum_{k=-\infty}^{\infty} u_k\, e^{2\pi\, k\, \mathrm{i}\, x}$.

A3 [5 marks]

Define the function

$$\tilde{u}_J(x) := \sum_{k=-J}^{J} u_k\, e^{2\pi\, k\, \mathrm{i}\, x},$$

for u_k found in **A2**. Write a function *step_fn* to evaluate $\tilde{u}_J$ on a grid of 1000 uniformly spaced points on $[0, 1]$.

Plot $\tilde{u}_J(x)$ and the step function $u(x)$ on $[0, 1]$ for $J = 5, 10, 25, 50$. Include plots in your report. Qualitatively describe the main features of the approximation.

Familiarise yourself with the Gibbs phenomenon using an information retrieval method of your choice

Look up *Gibbs phenomenon* (either online or in a textbook). Describe the phenomenon and its appearance in your plots.

In [1]:
```
import numpy as np

def step_fn(J):

    # Fourier approx to step function with 2J +1 terms

    N = 1000
    grid = np.linspace(0, 1, N)

    # FILL IN to define uJ

    return grid, uJ

import matplotlib.pyplot as plt
#Use plt to plot uJ for J = 5, 10, 25, 50
```

A4 [5 marks]

Repeat **A2-A3** with $u(x) = x^2 - x$.

A5 [5 marks]

Suppose that $u(x)$ is continuously differentiable. Show that

$$|u_j| \leq C_1 \frac{1}{|j|}, \qquad j = \pm 1, \pm 2, \ldots,$$

for $C_1 := (1/2\pi)\big(|u(0) - u(1)| + \int_0^1 |u'(x)|\, dx\big)$. Hence, show that

$$\int_0^1 (u(x) - \tilde{u}_J(x))^2\, dx \leq 2\, C_1^2\, \frac{1}{J-1}.$$

If $u(x)$ is twice continuously differentiable and $u(0) = u(1)$ (we say u is periodic on $[0,1]$), show that

$$\int_0^1 (u(x) - \tilde{u}_J(x))^2\, dx \leq C_2\, \frac{1}{(J-1)^3},$$

for a constant C_2. Use this inequality to explain the differences in the quality of approximation observed in **A2-A4**.

Section B.

In this section, we explore the composite trapezium rule.

The trapezium rule is remarkably effective for periodic functions and this can be explained using Fourier series.

B1. [5 marks]

Suppose that $u : [0,1] \to \mathbb{R}$ is a periodic function on $[0,1]$ (so that $u(0) = u(1)$). Show that the composite trapezium rule with uniform spacing $h = 1/J$ for the integral $\int_0^1 u(x)\, dx$ can be written as

$$T(u) = \frac{1}{J} \sum_{j=0}^{J-1} u\Big(\frac{j}{J}\Big).$$

Implement this as a function *per_trap* in Python. You should be able to call the routine with

```
I = per_trap(lambda x: np.cos(2*np.pi*x), J)
```

to evaluate $\int_0^1 \cos(2\pi x)\, dx$.

In [2]:
```
def per_trap(u,J):
    #FILL IN
    return I
```

For $u(x) = e^x$ and $u(x) = 1/(2 - \cos(2\pi x))$, compare the trapezium approximation to the integral $\int_0^1 u(x)\, dx$ with the exact value for a suitable range of J. Plot the absolute error against J and describe the main features observed.

Hint: plot using `semilogy`

B2. [5 marks]

Evaluate $T(e^{2\pi\,k\,\mathrm{i}\,x})$ and $\int_0^1 e^{2\pi\,k\,\mathrm{i}\,x}\,dx$ for integers k. Hence, provide an expression for the error

$$T(u) - \int_0^1 u(x)\,dx,$$

when $u(x) = \sum_{k=-\infty}^{\infty} u_k\, e^{2\pi\,k\,\mathrm{i}\,x}$.

B3. [5 marks]

Explain the difference in the quality of the quadrature observed in **B1**.

Section C.

Section C is on image compression. Fourier series are a key tool in image analysis, for example, in reducing the size required to store an image (so-called *compression algorithms*)

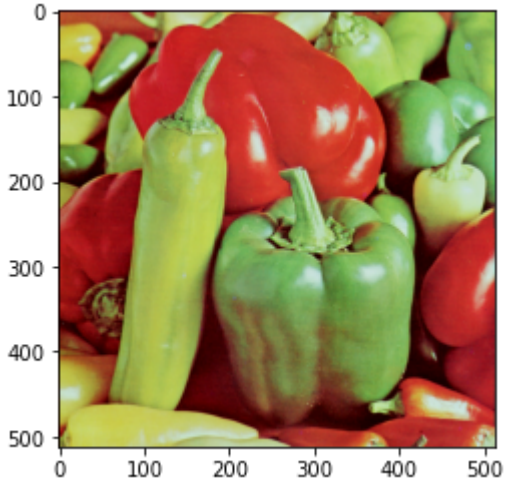
First choose a test image. Some examples are given in Moodle of olympians with their flag; the blocks of colour with edges on the flag and the fine details of the face make for good test cases; resolution 512x512 or above recommended.

Use the following commands, to load the image into Python:

In [3]:
```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

img = mpimg.imread('image_peppers.tiff')
imgplot = plt.imshow(img)
print('The size of img is', img.shape)
```

```
The size of img is (512, 512, 3)
```



You now have an $n \times m \times 3$ (colour) or an $n \times m$ (black and white) matrix.

If you have a colour image, try changing img to `img[:,:,i]` for $i = 0, 1, 2$ to look at the different colour channels.

Working with images that are $n \times m$ (i.e. black and white) is easier and recommended.

If you wish to persist with colours, do all operations on each slice `A = B[:,:,i]` of the image. However, **there are no extra marks** to be gained by working with colour images.

In the following, we treat A as an $n \times m$ matrix.

The command `numpy.fft.fft2` applies the fast Fourier transform $\mathcal{F}$ to an image A, and `numpy.fft.ifft2` applies the inverse fast Fourier transform $\mathcal{F}^{-1}$.

In Python, the output of `fft2` applied to an $m \times n$ image A is

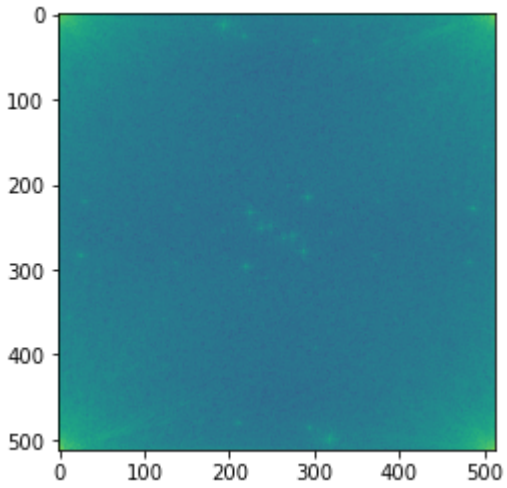
$$B_{p,q} = \sum_{j=0}^{m-1} \sum_{k=0}^{n-1} \exp\left(-\frac{2\pi i j p}{m}\right) \exp\left(-\frac{2\pi i k q}{n}\right) A_{j,k}\,,$$

where $p = 0, \ldots, m-1$ and $q = 0, \ldots, n-1$ and $A_{j,k}$ denotes the (j,k)-th entry of the matrix A.

Recall that $\mathcal{F}^{-1}(\mathcal{F}(A)) = A$. Note that due to this ordering, the coefficients corresponding to low frequencies are stored on the outer edges. These coefficients are typically the most significant ones. You can see this using `plt.imshow` as below:

In [5]:
```
import numpy as np
from numpy.fft import fft2, ifft2

fftim = fft2(img[:,:,2])
plt.imshow(np.log(np.abs(fftim))) # visualise absolute values of Fourier coefficients in log do
plt.show()
```



The command `fftshift` rearranges the coefficients such that the zero-th coefficient is stored in the centre:

In [6]:
```
from numpy.fft import fftshift, ifftshift

shiftim = fftshift(fftim)
plt.imshow(np.log(np.abs(shiftim))) # visualise absolute values of Fourier coefficients in log
plt.show()
```

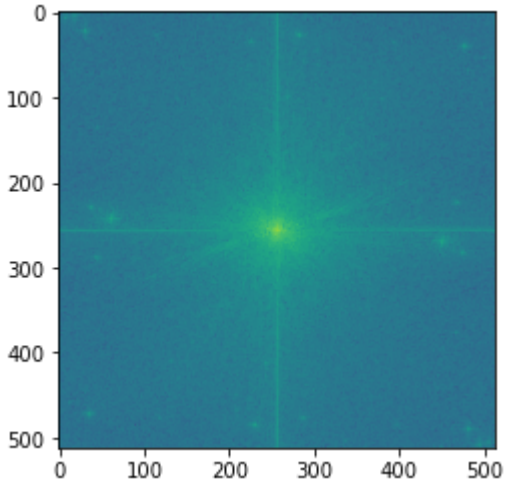


Image compression involves storing the important coefficients only — and accepting some approximation by discarding smaller coefficients.

A simple compression technique is to discard the higher frequency Fourier coefficients. That is, take $\hat{A}$ as an approximation to an image A with

$$\hat{A} = \mathcal{F}^{-1}(S(\mathcal{F}(A))$$

where the operator S discards some of the coeffients in $\mathcal{F}(A)$ by setting them to zero.

C1 [5 marks]

In this exercise, you will implement a compression procedure that retains only a fraction of the Fourier coefficients.

Write a routine that takes as input a matrix A, evaluates its Fourier transform, `A_hat = fft2(A)` (here, `A_hat` is denoted as $\hat{A}$), and store the most significant elements in a third matrix `C_hat` (i.e. $\hat{C}$).

Given J, the matrix $\hat{C}$ is the $n \times m$ matrix whose non-zero entries are given by $\hat{C}_{ij} = \hat{A}_{ij}$, for $i = 1, \ldots, J$ or $i = n - J + 1, \ldots, n$, and $j = 1, \ldots, J$ or $j = m - J + 1, \ldots, m$.

That is, the four corners of $\hat{C}$ are taken from $\hat{A}$; everything else is zero.

For $J < \min(n, m)/2$, the matrix $\hat{C}$ contains only $4J^2$ non-zero entries or pieces of data compared to nm pieces of data for A. In other words, $\hat{C}$ takes less storage than $\hat{A}$.

Finally, we apply the inverse transform `C = ifft2(hat_C)` , to recover an image from the compressed data $\hat{C}$.

In [7]:
```
def compress(A, J):
    # assume input is black-and-white or a slice A(:,:,i) of a rgb image
    A_hat = fft2(A)


    #FILL IN
    #Chat = #keep only most important  4J^2 coefficients in Ahat

    C = np.real(ifft2(Chat)) #only interested in real part
    return C
```

What is the quality of the compressed image?

Choose a test image and check your algorithm by comparing A and C, using `imshow`.

Your hand-in should include a test image of at least 512×512 resolution and give compressed images for four different values of J.

What artefacts do you observe?

C2. [5 marks]

Can you improve the compression algorithm?

Divide the image matrix into blocks and suppose that A is a $N \times M$ block matrix,

$$A = \begin{pmatrix} A_{11} & A_{12} & \dots A_{1M} \\ A_{21} & A_{22} & \dots A_{21M} \\ \vdots & & \\ A_{N1} & A_{N2} & \dots A_{NM} \end{pmatrix},$$

where each A_{ij} is also a matrix. Suppose that A_{ij} are $d \times d$ matrices for $i = 1, \dots, N-1$ and $j = 1, \dots, M-1$. The last row and column may contain matrices of a different size to accommodate your image.

Apply the Fourier-transform compression algorithm on each sub-matrix A_{ij} to find a new compressed image. Record how many values are stored for each sub-matrix and the total required for the whole image.

Compare four block sizes and choice of J and look for one that improves on the compression in **C1** (In a qualitative sense; so no need to compute any norms. Simply look and make your own subjective judgement).

Discuss your findings and any improvements or otherwise over **C1**.

In [8]:
```
#inputs: image im
#        J = coefficients in each block
#        D = size of blocks
def compress_v2(A, J, D):
    # FILL IN

    #return the compressed image as C and the total number of coefficients used to compute C
    return C, total_coefficients
```

A few lines of code to show you how to extract a section of a matrix:

In [9]:
```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import numpy as np

#import a colour image
img = mpimg.imread('image_peppers.tiff')
imgplot = plt.imshow(img)
plt.show()

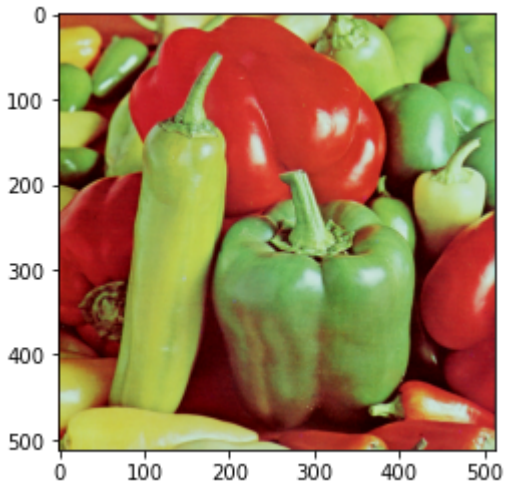
print('The size of img is', img.shape)

#extract the first 100 by 100 pixels of img
img_small = img[1:100, 1:100, :];
plt.imshow(img_small)
plt.show()
```

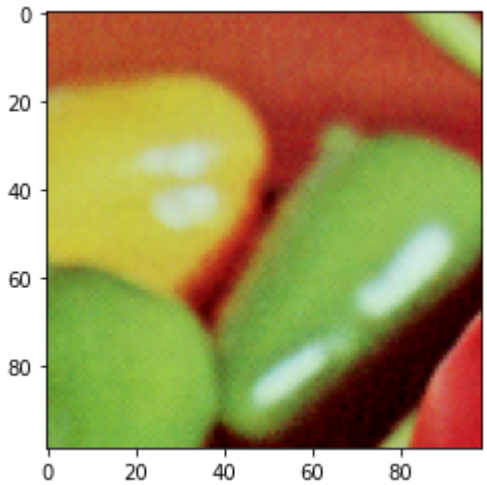
```
print('The size of img_small is', img_small.shape)


#create a zero matrix of the same dimensions as img and place img_small in the top left corner
img2 = np.zeros_like(img)
img2[1:100, 1:100, :] = img_small
plt.imshow(img2)
plt.show()

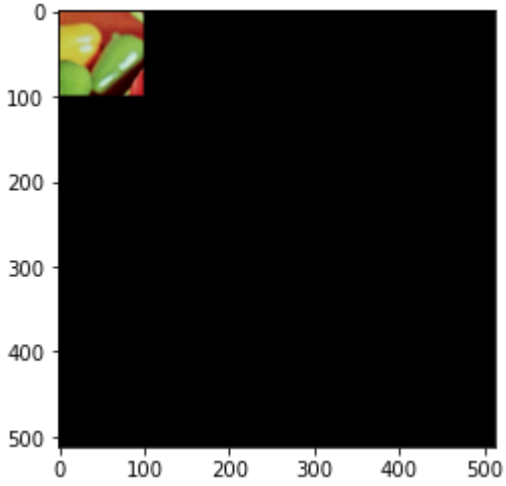
print('The size of img2 is', img2.shape)
```




```
The size of img is (512, 512, 3)
```




```
The size of img_small is (99, 99, 3)
```




```
The size of img2 is (512, 512, 3)
```